Code Seed

LECTURE 01 · FOUNDATIONS

How the Web Works

Clients, servers, and the language they speak: HTTP.

01

Understand the concept. **Then build it.**

THE CODE SEED WAY

Understand the concept.
Then build it.

Three short lectures first. Then we write every line of the app together — and you will know why each line exists.

THE PROJECT

In our Knowledge Base

A Personal Knowledge Base

Capture

Notes, code snippets, technical concepts

Organize

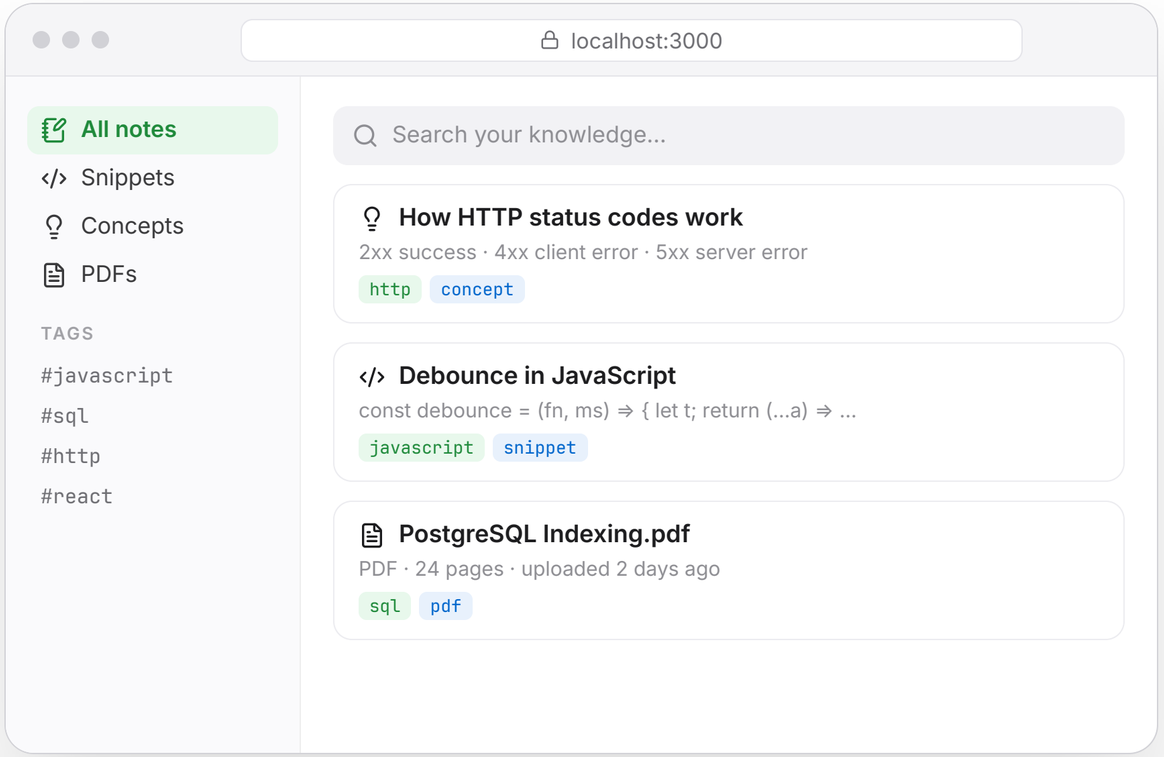
Tags that connect related ideas

Upload

PDFs — papers, docs, cheat sheets

Find

Search across everything you saved



THE FOUNDATIONS

Three ideas before we build

YOU ARE HERE

LECTURE 01

How the Web Works

Client · Server · HTTP

LECTURE 02

APIs & REST

Endpoints · CRUD · JSON

LECTURE 03

Databases

Tables · Relations · SQL · Search

Then → the coding series

THINK ABOUT IT

What actually happens when you click Save?

Between your click and "Saved ✓" — what travels where, and who decides it worked?
By the end of this lecture, you can trace every step.

01

The Big Picture

Two computers, one conversation.

CLIENT–SERVER

Two roles, one conversation

Client

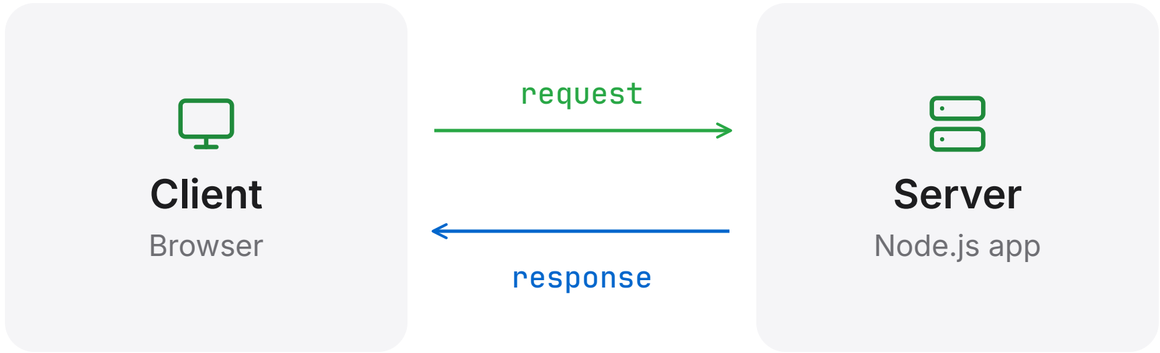
Asks for things. Your browser, a phone app.

Server

Answers. A program on a computer that is always on.

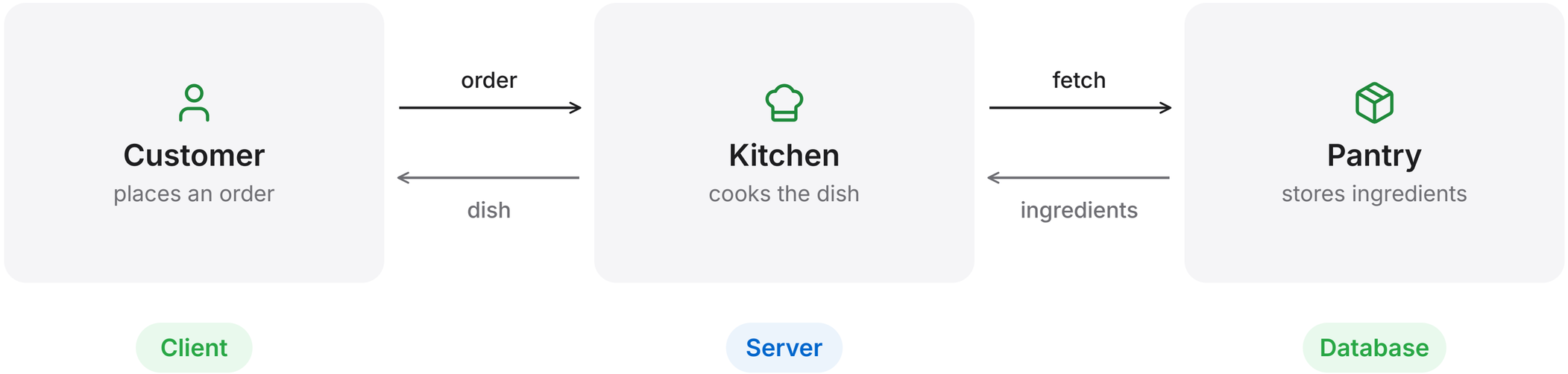
The rule

The client **requests**. The server **responds**.



ANALOGY

Think of a restaurant



You never walk into the kitchen. You place an order — and wait for a response.

THE CLIENT

In our Knowledge Base

The client: what users touch

Renders the interface

HTML for structure, CSS for style, JS for behavior

Captures input

Typing a note, picking tags, choosing a PDF

Sends requests

...and shows whatever comes back

Is never trusted

Anyone can edit code running in their own browser



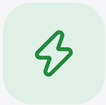
HTML

Structure — the note editor, the list



CSS

Style — layout, fonts, dark mode



JavaScript

Behavior — click Save → send request

In our Knowledge Base

The server: where rules live

Always on

Listens for requests, day and night

Enforces rules

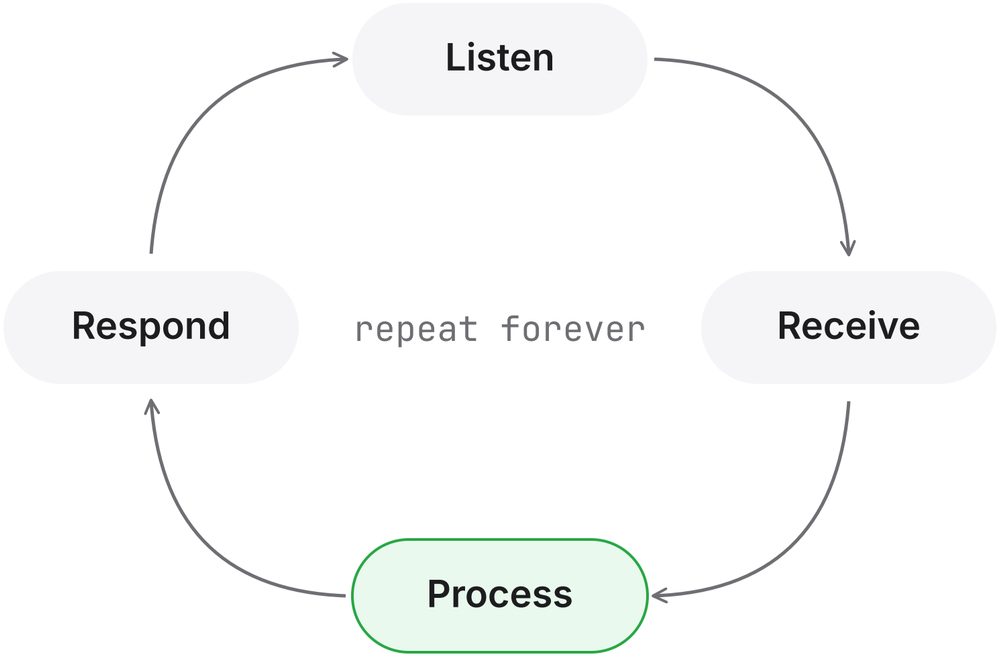
Is there a title? Is this your note?

Talks to storage

Database for data, disk or cloud for PDFs

Responds

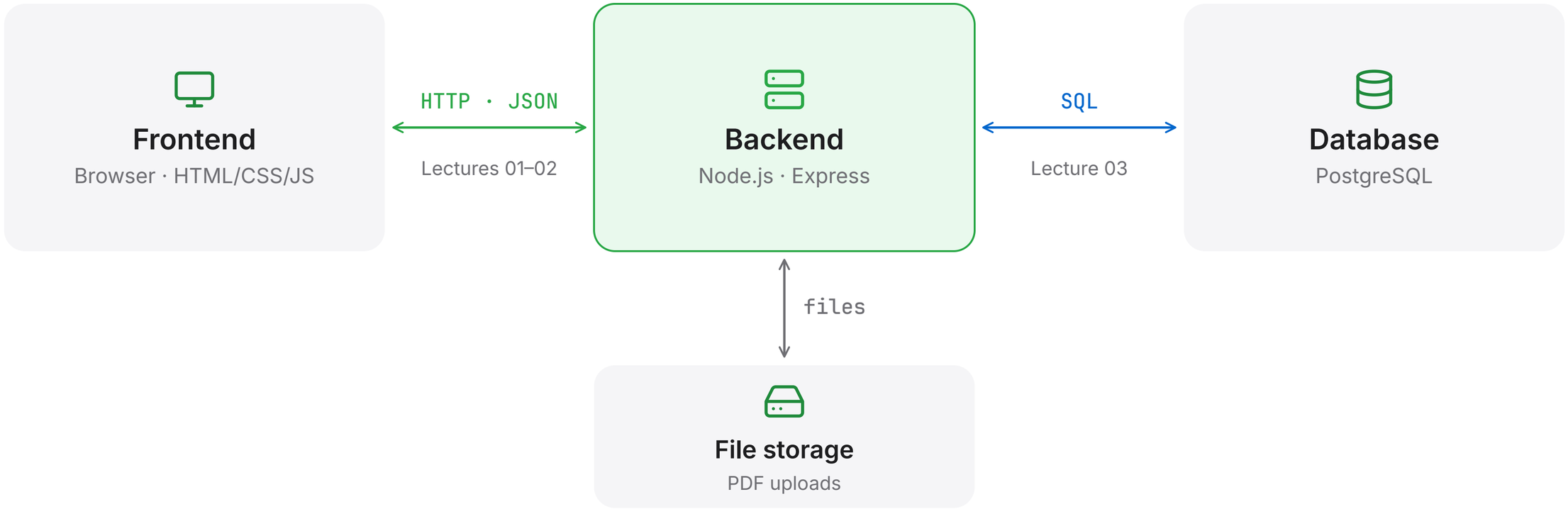
With data — or with a clear error



ARCHITECTURE

In our Knowledge Base

Our app, in three tiers



Each tier has one job. Each foundation lecture explains one connection.

DESIGN QUESTION

In our Knowledge Base

Why not keep it all in the browser?

✕ BROWSER ONLY

localStorage

- Gone when the cache is cleared
- Stuck on one device
- About 5 MB — PDFs won’t fit
- No backups, no accounts

✓ WITH A SERVER

Client + server

- Survives any browser or device
- Same notes on laptop and phone
- Gigabytes of PDFs in storage
- Backups, search, user accounts

02

Finding the Server

Addresses, names, and ports.

ADDRESSES

In our Knowledge Base

Anatomy of a URL

`https://kb.example.com:443/notes/42?tag=sql`

Scheme
Which protocol — secure HTTP

Host
Which machine (domain name)

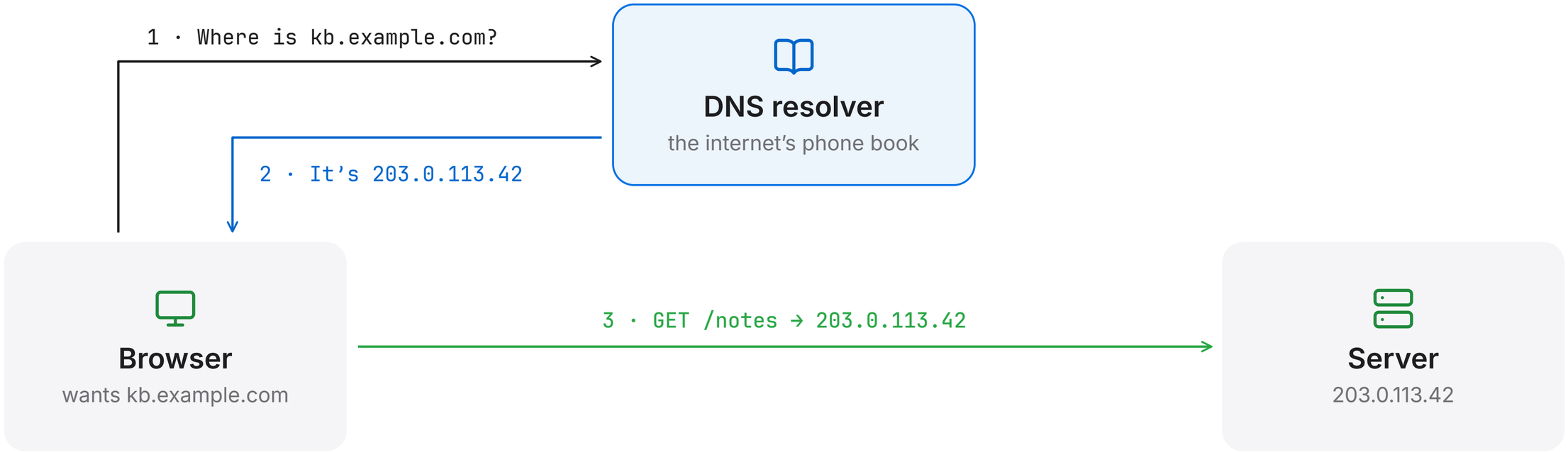
Port
Which program — usually hidden

Path
Which resource on the server

Query
Extra options: filter, search, sort

DNS

DNS: names into numbers



Humans remember names. Networks route by numbers. DNS translates — and caches the answer.

PORTS

One machine, many doors

IP address

Finds the machine

Port

Finds the program on that machine

Well-known ports

`80` HTTP · `443` HTTPS · `5432` PostgreSQL

In development

`localhost:3000` — your own computer

server.js JAVASCRIPT

```
// server.js
import express from 'express';

const app = express();

app.listen(3000, () => {
  console.log('Listening on http://localhost:3000');
});
```

03

HTTP

The language of the web.

HTTP

The request–response cycle



One request, one response. The client always speaks first.

HTTP REQUEST

Anatomy of a request

raw request HTTP

```
POST /api/notes HTTP/1.1

Host: kb.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi…

{
  "title": "HTTP status codes",
  "body": "2xx means success…",
  "tags": ["http", "concepts"]
}
```

Request line
Method · path · version

Headers
Metadata about the request

Body
The data itself — here, JSON

HTTP RESPONSE

Anatomy of a response

raw response — HTTP

```
HTTP/1.1 201 Created

Content-Type: application/json
Location: /api/notes/42

{
  "id": 42,
  "title": "HTTP status codes",
  "createdAt": "2026-10-31T09:30:00Z"
}
```

Status line

Version · code · reason

Headers

Format, location of the new note

Body

The saved note — now with an id

HTTP METHODS

In our Knowledge Base

Methods: the verb of a request

METHOD	MEANING	IN OUR KNOWLEDGE BASE
`GET`	Read data	Load all notes, open one note
`POST`	Create something new	Save a new note, upload a PDF
`PUT`	Replace it entirely	Overwrite a whole note
`PATCH`	Change part of it	Rename a note, pin it
`DELETE`	Remove it	Delete a note or a tag

STATUS CODES

Status codes: answers in 3 digits

2xx · Success

It worked.

```
200 OK
201 Created
204 No Content
```

3xx · Redirect

Look somewhere else.

```
301 Moved Permanently
304 Not Modified
```

4xx · Client error

You sent something wrong.

```
400 Bad Request
401 Unauthorized
404 Not Found
```

5xx · Server error

We broke something.

```
500 Internal Error
503 Unavailable
```

THINK ABOUT IT

Which status code would you send?

1. A note was saved successfully
2. Someone requests note #999 — it doesn’t exist
3. A user submits a note with no title
4. The database crashes mid-request

Headers: metadata for the message

HEADER	ANSWERS THE QUESTION	EXAMPLE
`Content-Type`	What format is the body?	`application/json`
`Authorization`	Who is asking?	`Bearer <token>`
`Accept`	What format do I want back?	`application/json`
`Cache-Control`	Can this response be reused?	`no-store`

DATA FORMAT

In our Knowledge Base

JSON: the shape of our data

Plain text

Every language can read and write it

Simple building blocks

Objects `{}` · arrays `[]` · strings · numbers · booleans · `null`

Our note

Exactly this travels between browser and server

note.json JSON

```
{
  "id": 42,
  "type": "snippet",
  "title": "Debounce in JavaScript",
  "language": "javascript",
  "tags": ["javascript", "performance"],
  "pinned": false,
  "updatedAt": "2026-10-31T09:30:00Z"
}
```

STATELESS

HTTP has no memory

Stateless

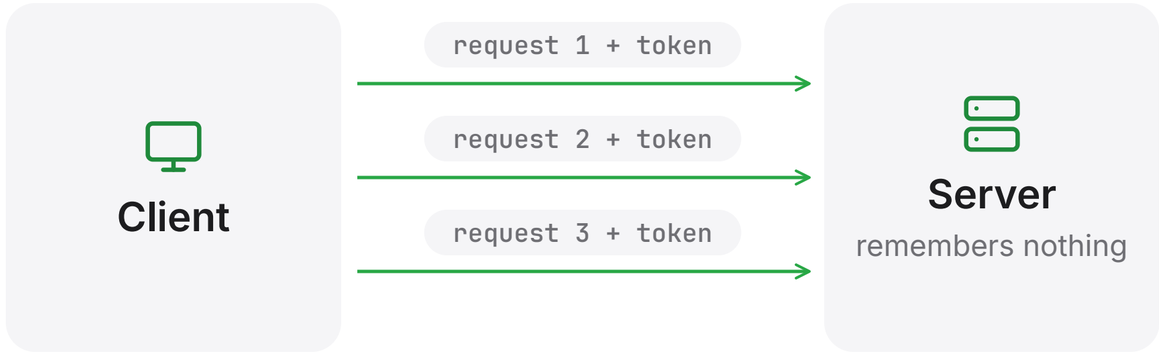
Each request stands alone. The server forgets you after replying.

So requests carry context

Who you are travels as a token in `Authorization`

Why that is good

Simpler, and any server can answer any request



SECURITY

In our Knowledge Base

HTTPS: the S means secure

Encrypted

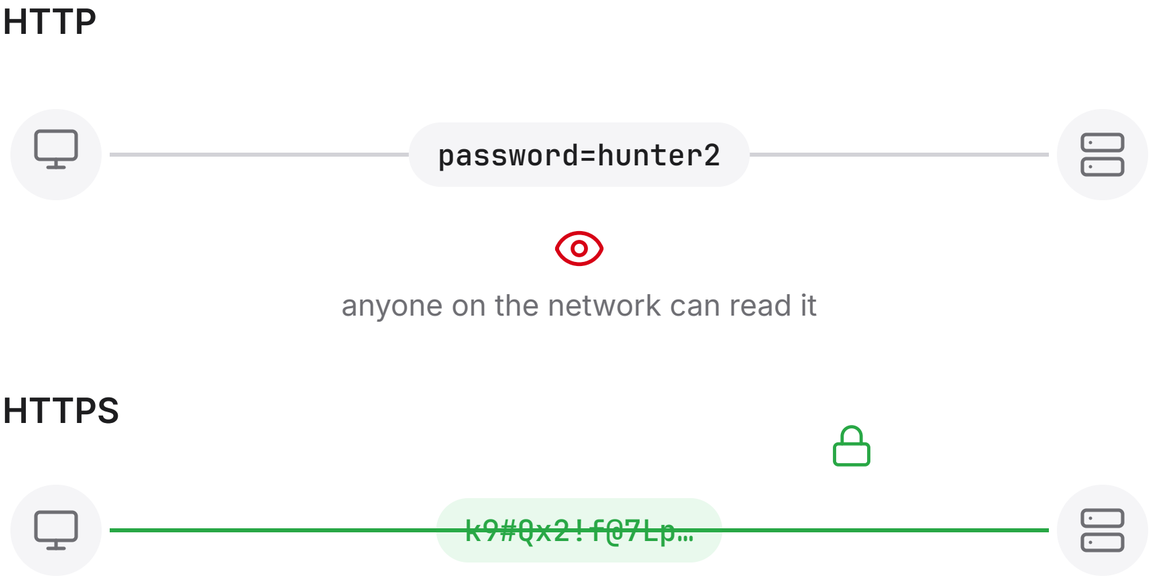
Nobody on café Wi-Fi can read your notes

Authentic

A certificate proves it is the real server

Non-negotiable

Logins and private notes require it



04

See It in Action

From click to “Saved ✓”.

CLIENT CODE

The client sends a request

app.js — JAVASCRIPT

```javascript
// Runs in the browser when the user clicks "Save"
async function saveNote(note) {
  const res = await fetch('/api/notes', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(note),
  });

  if (!res.ok) throw new Error(`Save failed: ${res.status}`);
  return res.json();   // → { id: 42, title: … }
}
```

`fetch()`
The browser’s built-in HTTP client

`method` + `headers`
Request line and metadata

`body`
Our note, as JSON text

`res.status`
The 3-digit answer

SERVER CODE

The server responds

server.js JAVASCRIPT

```
import express from 'express';
const app = express();
app.use(express.json());              // parse JSON bodies

app.post('/api/notes', (req, res) => {
  const { title, body } = req.body;
  if (!title) {
    return res.status(400).json({ error: 'Title is required' });
  }
  const note = { id: 42, title, body };  // database: Lecture 03
  res.status(201).json(note);
});

app.listen(3000);
```

Route

Method + path → a function

`req`

Everything the client sent

`res`

How we answer: status + JSON

Validate first

Never trust the client

FULL TRACE

In our Knowledge Base

Click “Save” — what really happens

01 **Click**
User presses Save in the browser

02 **fetch()**
JavaScript builds a POST request with a JSON body

03 **DNS + connect**
Find the server, open a secure TLS connection

04 **Route**
Server matches `POST /api/notes`

05 **Validate**
Title present? Is the user logged in?

06 **Store**
Insert into the database — Lecture 03

07 **Respond**
`201 Created` + the saved note as JSON

08 **Update UI**
Browser shows “Saved ✓”

TOOLS

Watch it happen: the Network tab

Elements Console Network Sources Application

NAME	METHOD	STATUS	TYPE	SIZE	TIME
notes	GET	200	fetch	4.2 kB	38 ms
notes	POST	201	fetch	512 B	54 ms
notes/999	GET	404	fetch	89 B	21 ms
tags	GET	200	fetch	1.1 kB	17 ms
search?q=index	GET	500	fetch	120 B	9 ms

Open DevTools on any site: ⌥⌘I on Mac · F12 or Ctrl+Shift+I on Windows

RECAP

Lecture 01 in six ideas

01 **Client asks, server answers**
Browser ↔ an always-on program

02 **Three tiers**
Frontend · Backend · Database + file storage

03 **URLs and DNS find the server**
Names → IPs, ports → programs

04 **HTTP is the language**
Method, path, headers, body → status, body

05 **Stateless and secure**
Every request carries context, over HTTPS

06 **JSON is the payload**
Our notes travel as plain JSON text



NEXT · LECTURE 02

APIs & REST

We know how browser and server talk. Next: designing what they say — the API for our Knowledge Base.

Homework: open the Network tab on any site and read five requests